

Big Data y Machine Learning para Economía Aplicada

Week 03: Validación cruzada y regularización — el *pipeline* honesto

Eduard F. Martinez Gonzalez, Ph.D.

Departamento de Economía, Universidad Icesi

September 18, 2026

Previously...

- **El caso del curso:** predecir el voto por Petro en cada puesto de votación (segunda vuelta de 2022) con el censo de su entorno. En Cali, Palmira, Yumbo y Jamundí una recta con la educación superior dejó el RMSE de prueba en 6,8 puntos (la media, 11,0).
- **OLS como máquina de predecir:** mismas fórmulas, otro criterio. El intervalo que importa es el de predicción, que no se cierra con datos; la tabla canónica siempre lleva la referencia en la primera fila.
- Dos decisiones quedaron tomadas **a ojo**: el k de los vecinos (5) y qué variables entran (una o nueve). Y una prueba de 86 puestos que miramos ocho veces.
- **La limitación:** la recta no viaja. Fuera de las cuatro ciudades predice peor que la media local, y en el país explica el 0,04% del voto.

La tabla del líder, al cierre de la semana 2 (área metropolitana, 86 puestos)

Media 11,0 · el voto de 2018 corregido 8,4 · k -NN en el mapa 8,2 · **recta 6,8** · nueve variables 6,8.

La pregunta de hoy: 37 predictores, ¿cuáles y cómo saberlo?

- De los conteos del censo salen **37 predictores**: educación, edades, etnia, servicios, siete estratos, tamaño del entorno. Si cada uno pesa distinto en cada departamento, **1.253 columnas**.
- **La pregunta**: ¿cuáles usamos? Y antes: ¿cómo sabemos que una decisión (añadir variables, cambiar k , penalizar) **mejora la predicción fuera de muestra**, sin gastar la prueba cada vez que dudamos?

Dos actos, el mismo caso:

- 1 **Pocos puestos, muchos predictores**: los 257 puestos de entrenamiento del área metropolitana con la ficha completa. Aquí se aprende a elegir.
- 2 **El país**: 9.603 puestos para aprender y 2.398 bajo llave, la partición fija del curso. Aquí se estrena la tabla del líder nacional.

El plan

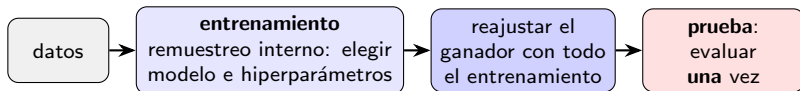
Primero la teoría, con estos puestos como ejemplo (validación cruzada, el *pipeline* sin filtraciones, grupos y tiempo, Ridge, Lasso y Elastic Net); después, lo mismo en R.

- 1 Elegir sin gastar la prueba: la validación cruzada
- 2 El *pipeline* honesto: preprocesar dentro de cada pliegue
- 3 Cuando el k -fold aleatorio miente: grupos, espacio y tiempo
- 4 Muchos predictores: Ridge, Lasso y Elastic Net
- 5 Aplicación guiada en R
- 6 Cierre

Dos preguntas, un solo conjunto de prueba

Necesitamos el error fuera de muestra para dos cosas distintas:

- 1 **Elegir:** ¿qué variables? ¿qué k ? ¿qué λ (hoy)? ¿qué profundidad (sesión 5)? Muchas comparaciones.
- 2 **Reportar:** cuánto se equivoca el modelo final en puestos nuevos. Una sola cifra.



- La prueba sólo puede responder la pregunta 2, y una sola vez (regla de oro #2). Si la usáramos para elegir entre veinte configuraciones, dejaría de ser prueba: sería un segundo entrenamiento. La semana pasada la miramos ocho veces.
- La pregunta 1 se responde **dentro del entrenamiento**, remuestreándolo. De eso trata la primera mitad de hoy.

El punto de partida: el conjunto de validación

Apartar una fracción del entrenamiento y evaluar ahí (ISL, sección 5.1.1)



Con los 257 puestos: 193 para ajustar tres modelos (la recta, las nueve variables, la ficha de 37) y 64 para validar. Repetido con cinco particiones al azar, ganó **la recta, la ficha, la ficha, las nueve y las nueve**; y el error de un mismo modelo fue de 5,9 a 8,4 según la partición.

Funciona, pero tiene dos defectos:

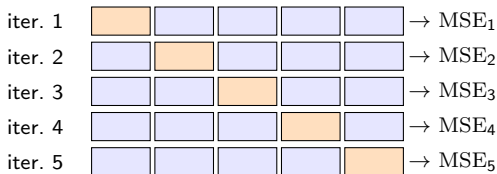
- 1 **Alta variabilidad:** el error estimado depende de *cuáles* puestos cayeron en validación. Otra partición, otro número y a veces otro ganador.
- 2 **Desperdicia datos:** el modelo se entrena con una parte de la muestra; con menos datos aprende peor $\Rightarrow$ tiende a **sobreestimar** el error del modelo entrenado con todo.

La idea de la validación cruzada

Reutilizar **todos** los datos del entrenamiento: que cada puesto sea validación *alguna vez* y entrenamiento *el resto* de las veces.

k-fold: el estándar

Partir el entrenamiento en k bloques (pliegues) de tamaño similar; cada bloque es validación una vez:



$$CV_{(k)} = \frac{1}{k} \sum_{j=1}^k MSE_j, \quad \widehat{EE} = \frac{\text{desv. est.}(MSE_1, \dots, MSE_k)}{\sqrt{k}}$$

- Cada modelo entrena con la fracción $(k - 1)/k$ de los datos; se ajusta k veces. **Estándar:** $k = 5$ o 10 . Con 257 puestos y $k = 5$: cinco pliegues de 51 o 52; cada puesto es validación una vez y entrenamiento cuatro.
- Los k errores dan, además del promedio, una **medida de incertidumbre**: es lo que las barras de la curva de CV van a mostrar, y lo que usa la regla 1-SE.

Leave-one-out: el caso límite

Con $k = n$: dejar **un solo** puesto afuera, entrenar con los otros $n - 1$, predecirlo; repetir para cada i :

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{(-i)})^2, \quad \hat{y}_{(-i)} : \text{predicción de } i \text{ sin usar } i.$$

- **A favor:** cada modelo usa $n - 1$ datos, así que casi no hay sesgo; y no hay azar.
- **En contra:** n ajustes (para OLS hay un atajo con un solo ajuste, en el apéndice; para bosques y redes, no). Y **varianza alta:** los n modelos son casi idénticos, y promediar errores muy correlacionados reduce poco la varianza.

La lección del promedio correlacionado

Promediar n cosas sólo ayuda si son *distintas* entre sí. Vuelve en la sesión 5 con los bosques aleatorios.

¿Qué k ? El sesgo–varianza del propio CV

El estimador CV es, él mismo, una variable aleatoria: tiene sesgo y varianza.

	Sesgo	Varianza	Costo
Conjunto de validación	alto (pesimista)	alta	1 ajuste
LOOCV	≈ 0	alta	n ajustes
k -fold ($k = 5, 10$)	bajo	moderada	k ajustes

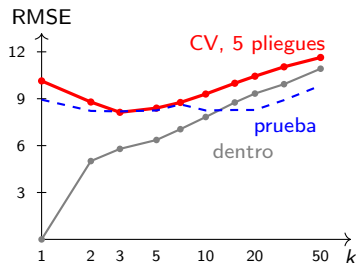
- **Sesgo:** entrenar con menos de n datos hace ver los modelos peores de lo que serán: el CV es algo *pesimista*, más con k pequeño y con modelos grandes. Con la ficha de 37: 7,8 con 5 pliegues (205 puestos por ajuste) y 7,2 con LOOCV (256). Por eso el ganador se **reajusta** con todo.
- **Varianza:** LOOCV promedia errores de modelos casi clones; k -fold usa modelos más distintos entre sí. **Veredicto:** $k = 10$ por defecto, $k = 5$ si el ajuste es costoso; repetir el CV con varias particiones si n es pequeño.

En clasificación (sesión 4)

Pliegues **estratificados**: cada uno conserva la prevalencia (con 2% de positivos, un pliegue sin positivos no mide nada). La métrica se calcula **por pliegue** y luego se promedia.

La curva de CV: girar la perilla y mirar

k -NN en el mapa con los 257 puestos: RMSE dentro, de CV y de prueba para cada k



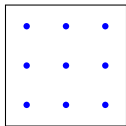
- **Dentro** (gris): con $k = 1$ el error es 0 (cada puesto es su propio vecino) y sube con k . Regla de oro #1: aquí la complejidad crece hacia la *izquierda*.
- **CV** (rojo) tiene forma de U: 10,1 con $k = 1$, **8,1** con $k = 3$, 11,6 con $k = 50$.
- **Prueba** (azul): 8,9, 8,2, ..., 9,8. El CV anticipó dónde estaba el valle sin abrirla.

Es la curva U de la sesión 1, ahora estimada con datos

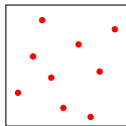
El eje horizontal puede ser cualquier perilla: el k de los vecinos, el número de variables, el λ de hoy, la profundidad de un árbol. El eje vertical es siempre el mismo: el error en puestos que el modelo no vio.

Elegir con la curva: el mínimo, la regla 1-SE y cómo buscar

hiperparámetro 2



grilla 3×3



aleatoria, 9 puntos
hiperparámetro 1

- **El mínimo** de la curva: $k = 3$ en los vecinos.
- **Regla 1-SE**: el modelo **más simple** cuyo CV está a menos de un error estándar del mínimo. El propio CV es ruidoso ($8,1 \pm 0,6$): ante un empate estadístico, parsimonia. Con k -NN, “más simple” es k más grande: $k = 5$, el que la semana pasada pusimos a ojo.
- **Grilla**: 3 hiperparámetros $\times$ 10 valores = 1.000 configuraciones, cada una con k ajustes. No escala. **Aleatoria** (Bergstra & Bengio, 2012): con el mismo presupuesto explora más valores de cada uno; gana cuando sólo uno o dos importan. **Bayesiana** (Snoek et al., 2012): un modelo del CV decide dónde probar (apéndice).
- **Cuidado**: el mejor CV entre cien configuraciones es **optimista** (el máximo de cien variables ruidosas). El número que se reporta sale de la prueba, o de un CV anidado (apéndice).

- 1 Elegir sin gastar la prueba: la validación cruzada
- 2 El *pipeline* honesto: preprocesar dentro de cada pliegue
- 3 Cuando el k -fold aleatorio miente: grupos, espacio y tiempo
- 4 Muchos predictores: Ridge, Lasso y Elastic Net
- 5 Aplicación guiada en R
- 6 Cierre

La trampa del *leakage*: el CV mal hecho

Una columna que parece una buena idea: el voto promedio del municipio

Escenario: los 9.603 puestos de entrenamiento del país y la ficha de 37. Un puesto de Cali se parece a los demás puestos de Cali; añadamos a cada puesto el **voto promedio de su municipio**.

Procedimiento (¿ven el error?):

- 1 Con **todos** los puestos de entrenamiento, calcular el voto promedio de cada municipio y pegarlo como columna.
- 2 Con la ficha y esa columna, ajustar OLS y estimar su error por CV de 5 pliegues.

La trampa del *leakage*: el CV mal hecho

Una columna que parece una buena idea: el voto promedio del municipio

Escenario: los 9.603 puestos de entrenamiento del país y la ficha de 37. Un puesto de Cali se parece a los demás puestos de Cali; añadamos a cada puesto el **voto promedio de su municipio**.

Procedimiento (¿ven el error?):

- 1 Con **todos** los puestos de entrenamiento, calcular el voto promedio de cada municipio y pegarlo como columna.
- 2 Con la ficha y esa columna, ajustar OLS y estimar su error por CV de 5 pliegues.

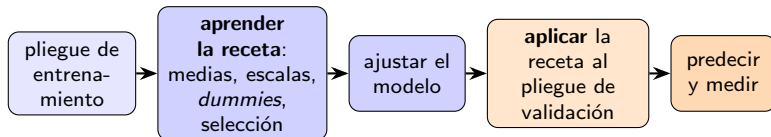
Resultado: el CV reporta un RMSE de **11,3**: el mejor modelo que hemos visto (la ficha con departamentos da 15,8). Pero cada puesto **entró en su propio promedio**: el paso 1 ya “vio” las respuestas de todos los pliegues. Hecho bien (el promedio de cada municipio calculado *dentro* de cada pliegue, sin los puestos que se van a predecir), el CV es **13,3**; y la prueba, que nadie tocó, dice **13,0**. La trampa prometía dos puntos que no existen.

La regla: todo paso que use *y*, o que aprenda algo de los datos (agregados por grupo, selección de variables, imputación, estandarización, remuestreo por desbalance, PCA), ocurre **dentro** de cada pliegue, sólo con sus datos de entrenamiento.

Regla de oro #3

El CV no valida modelos: valida **pipelines completos**.

Anatomía de un *pipeline*



- La **receta** es todo lo que transforma las x antes del modelo. Se *aprende* (las medias para imputar, las desviaciones para escalar, el promedio de cada municipio) en el entrenamiento del pliegue y se *aplica* tal cual a la validación. En nuestra aplicación, el promedio municipal se calcula cinco veces, una por pliegue.
- Lo mismo vale para el conjunto de prueba al final: la receta aprendida con todo el entrenamiento se aplica a la prueba. Nunca al revés. Y un municipio que no estaba en el entrenamiento (35 puestos de la prueba) recibe el promedio nacional: la receta también dice qué hacer con lo nuevo.

Lo que cambia de aquí al final del curso

Sólo la caja “ajustar el modelo”: hoy OLS, k -NN y `glmnet`; luego logística, bosques, *boosting*, redes. La receta y el remuestreo son los mismos.

Ingeniería de variables (I): las columnas que uno construye

La receta también *crea* predictores; cada uno es una perilla más

Columna nueva	En nuestro caso	Qué le da al modelo
Proporciones	de conteos del censo a porcentajes del entorno	comparar puestos de distinto tamaño
Transformaciones	log(registrados), log(población)	colas largas domadas
Razones	personas por hogar, hogares por vivienda	lo que la economía dice que importa
Interacciones	educación $\times$ rural; la ficha $\times$ departamento	efectos que dependen de otra variable
Agregados por grupo	el voto promedio del municipio	contexto que el puesto no trae
Rezagos	el voto de 2018 en el mismo puesto	memoria
Codificaciones	<i>dummies</i> de departamento; Vaupés y Guainía $\rightarrow$ "otros"	que lo cualitativo entre a la fórmula

- Con 37 variables base y 33 departamentos, dejar que cada una pese distinto en cada departamento da $37 + 32 + 37 \times 32 = 1.253$ columnas. Con 9.603 puestos OLS todavía existe; veremos qué hace.
- **Regla práctica:** construyan lo que la economía sugiera, y dejen que el **CV** (hoy) o la **penalización** (en una hora) decidan qué se queda. Los árboles (sesión 5) descubren interacciones solos.

Ingeniería de variables (II): lo que se aprende de los datos, se aprende en el pliegue

Paso	Qué aprende de los datos	Si se hace con toda la muestra. . .
Imputar faltantes	la media o mediana de cada x	filtración leve
Estandarizar	media y desviación de cada x	leve; <code>glmnet</code> y k -NN dependen de la escala
Agrupar categorías raras	qué categorías tienen pocos casos	leve
Agregados por grupo con y	el voto medio del <i>municipio</i>	grave: cada puesto entra en su propio promedio
Seleccionar variables	qué columnas se correlacionan con y	grave: con muchas columnas, se elige ruido que “predice” (ESL, 7.10.2)
Remuestrear por desbalance	duplica positivos	grave: copias en entrenamiento y validación (sesión 4)

- El criterio es siempre el mismo: ¿este paso **usa** y o **usa observaciones** que luego serán de validación? Si sí, ocurre dentro del pliegue.
- Las columnas que no aprenden nada (un logaritmo, una proporción, una razón) se pueden construir antes de partir: no hay nada que filtrar.

Otras formas de filtración

Cuatro filtraciones menos visibles que la del promedio municipal, frecuentes en datos económicos:

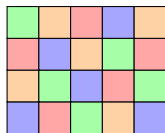
- 1 **Variables que contienen a y:** los votos por Petro o el total de votos del puesto al predecir su porcentaje; la fecha de castigo al predecir el impago; el precio por metro cuadrado al predecir el precio. Se delatan por métricas demasiado buenas.
- 2 **Información del futuro:** usar el resultado de las elecciones locales de 2023 para “predecir” 2022. Toda variable debe existir en el momento de decidir.
- 3 **Duplicados y grupos:** el mismo puesto registrado con dos nombres cae en entrenamiento y en prueba; los puestos de un mismo municipio se reparten entre pliegues. El modelo recuerda, no generaliza (siguiente parte).
- 4 **Afinar con la prueba:** elegir el modelo mirándola y luego reportarla.

El protocolo es parte del resultado

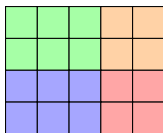
Una ventaja de dos puntos de RMSE puede ser entera de protocolo, como la nuestra (véase Neunhoeffter & Sternberg, 2019). Por eso el protocolo se reporta, no se supone.

- 1 Elegir sin gastar la prueba: la validación cruzada
- 2 El *pipeline* honesto: preprocesar dentro de cada pliegue
- 3 Cuando el k -fold aleatorio miente: grupos, espacio y tiempo
- 4 Muchos predictores: Ridge, Lasso y Elastic Net
- 5 Aplicación guiada en R
- 6 Cierre

Datos en grupos y en el espacio: vecinos que se copian



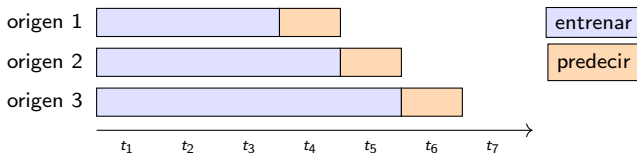
k-fold aleatorio



bloques (municipios)

- Los puestos de un mismo municipio comparten lo que el modelo **no observa**: la maquinaria local, la historia, el alcalde. Si uno cae en entrenamiento y otro en validación, el modelo *interpola* entre parientes y el CV luce mejor de lo que es.
- **CV por bloques** (Roberts et al., 2017): los pliegues son municipios enteros; cada pliegue es un “lugar nuevo”.
- **Con los 9.603 puestos del país**: k -NN en el mapa ($k = 10$) tiene un RMSE de CV de **12,3** con pliegues al azar y de **15,3** dejando fuera municipios enteros: tres puntos de su ventaja eran vecinos del mismo municipio. La ficha con departamentos pasa de 15,8 a 17,1: también copiaba, menos.
- El mismo principio aplica a cualquier **grupo**: se deja fuera el grupo entero, y las variables que lo identifican (la *dummy* de un departamento que nunca se vio, como Bogotá) dejan de servir.

Datos en el tiempo: validar hacia adelante



- Con datos en el tiempo **no se barajan** los pliegues: barajar deja que el futuro entrene al pasado. **Origen rodante:** entrenar hasta t , predecir $t + h$; avanzar el origen; promediar.
- **Nuestro caso:** Petro también estuvo en la segunda vuelta de 2018. Un modelo entrenado con los puestos de 2018 y aplicado a la prueba de 2022 tiene un RMSE de **19,9**; el entrenado en 2022, **16,1**. Corrigiendo sólo el nivel (Petro creció 9,4 puntos), 17,8.

Deriva y cambio de régimen

Predecir *otra* elección es extrapolar en el tiempo: la relación entre el censo y el voto se movió entre 2018 y 2022, y parte de lo que cambió fue el nivel, que ningún censo predice. Un modelo en producción se monitorea.

¿Qué pregunta responde cada esquema?

Esquema	Pregunta que responde	Cuándo es la correcta
<i>k</i> -fold aleatorio	¿qué tan bien predigo puestos nuevos en municipios ya vistos?	interpolan en la misma población
Por municipio	¿qué tan bien predigo en un municipio que el modelo nunca vio?	extrapolan a lugares nuevos
Por grupos	¿qué tan bien predigo para una unidad nueva (edificio, hogar, colegio)?	datos jerárquicos o de panel
Hacia adelante	¿qué tan bien predigo la próxima elección?	pronosticar

- Bloquear obliga a *extrapolar*: el error es honesto para esa tarea y pesimista para la otra. Una campaña que completa el mapa donde ya tiene datos necesita el primer número; una que entra a municipios nuevos, el segundo.
- **En el Problem Set 1** (vivienda en Cali): CV aleatoria contra CV por zonas; la diferencia es un resultado, no un detalle.

Regla

Elijan el esquema de validación según la **predicción que van a hacer**, y digan cuál eligieron.

Bootstrap: intervalos para las métricas, en una diapositiva

Otra pregunta: ¿qué tan **incierto** es un estadístico sin fórmula de error estándar? Un R^2 de prueba, una AUC, la diferencia de RMSE entre dos modelos.

El principio plug-in (Efron, 1979): la muestra es nuestra mejor estimación del mundo, así que remuestreamos de ella. Extraer B **remuestras con reemplazo** de tamaño n , calcular el estadístico $\hat{\theta}^{*b}$ en cada una, y usar la dispersión de las B réplicas:

$$\widehat{\text{EE}}(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_b (\hat{\theta}^{*b} - \bar{\theta}^*)^2}, \quad \text{IC}_{95\%} = [\hat{\theta}_{(0,025)}^*, \hat{\theta}_{(0,975)}^*].$$

- **Sobre la prueba:** remuestrear los puestos de prueba y recalcular la métrica; $B = 1.000$ suele bastar. Con 86 puestos, el RMSE de la recta (6,8) tiene un intervalo de 5,8 a 7,9, y la diferencia con la ficha $(-0,1)$ uno de $-1,1$ a $0,8$: no hay diferencia. Con 2.398, la ventaja del Lasso sobre la ficha con departamentos $(-0,9)$ va de $-1,1$ a $-0,7$: es real.
- **Cada remuestra deja afuera** cerca del 37% de las observaciones ($P(i \in \text{remuestra}) \rightarrow 1 - e^{-1} \approx 0,632$; la cuenta, en el apéndice): datos de validación *gratis* para el modelo ajustado en esa remuestra. Es el error *out-of-bag* de los bosques (sesión 5).

- 1 Elegir sin gastar la prueba: la validación cruzada
- 2 El *pipeline* honesto: preprocesar dentro de cada pliegue
- 3 Cuando el k -fold aleatorio miente: grupos, espacio y tiempo
- 4 Muchos predictores: Ridge, Lasso y Elastic Net
- 5 Aplicación guiada en R
- 6 Cierre

De dónde salen tantos predictores y qué falla

- **Datos administrativos y censos:** decenas de campos por unidad (la ficha de un puesto: 37 columnas). **La receta** multiplica las columnas (interacciones, *dummies*: 1.253). **Texto e imágenes** convertidos en variables: miles (sesión 7).

Tres regímenes: (1) p fijo y $n \rightarrow \infty$: Econometría I, OLS funciona. (2) p comparable a n : OLS “funciona” pero tiembla: varianzas enormes, coeficientes inestables. (3) $p > n$: OLS ni siquiera está definido de manera única; existen infinitos $\hat{\beta}$ con $R^2 = 1$.

Qué falla, exactamente: con predictores correlacionados, la varianza de OLS en la dirección v_j (autovector de $\mathbf{X}'\mathbf{X}$ con autovalor d_j) es σ^2/d_j . En las direcciones casi colineales (d_j pequeño) los datos casi no varían, casi no informan, y OLS adivina con enorme ruido.

Moraleja

El enemigo no es p grande *per se*: es la **información por parámetro**. Con 257 puestos y la ficha de 37, OLS baja el error dentro a 5,9 y su CV es 7,8, peor que la recta (7,4). Con 9.603 puestos y 1.253 columnas: 12,3 dentro y **118** en la prueba. Hoy reducimos el número *efectivo* de parámetros.

Seleccionar variables: subconjuntos, *stepwise* y criterios

ISL, sección 6.1, en una diapositiva

Pregunta previa: ¿por qué no comparar modelos por su R^2 ?

Seleccionar variables: subconjuntos, *stepwise* y criterios

ISL, sección 6.1, en una diapositiva

Pregunta previa: ¿por qué no comparar modelos por su R^2 ? Porque *siempre* mejora al agregar variables: es ajuste *dentro* de muestra. Comparar la recta con la ficha de 37 por R^2 (0,64 contra 0,76) es una carrera arreglada.

- **Mejor subconjunto:** probar los 2^p modelos. Con $p = 20$, un millón; con nuestros 37, $1,4 \times 10^{11}$. **Forward stepwise:** empezar vacío y agregar en cada paso la variable que más mejora el ajuste ($\approx p^2/2$ ajustes); **backward:** empezar completo e ir quitando. Ambos son *greedy*: no garantizan el mejor subconjunto.
- Entre el mejor modelo de 3 variables y el mejor de 8, ¿cuál gana? Hay que corregir el optimismo del error de entrenamiento: con un **criterio** o con **CV**.
- **Criterios** (C_p , AIC, BIC): error de entrenamiento + un castigo por parámetro, $C_p = \frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2)$. Nacieron cuando ajustar era caro y necesitan un d bien definido: ¿cuántos parámetros tiene un bosque? **Veredicto:** CV para todo; criterios como referencia rápida en modelos lineales.
- Y una decisión *discreta* (dentro o fuera) es **inestable**: pequeñas perturbaciones de los datos cambian el modelo entero. Guarden esta objeción.

Cambio de filosofía: de seleccionar a encoger

- **Seleccionar:** cada variable está dentro o fuera. Decisión discreta $\Rightarrow$ varianza alta.
- **Encoger** (*shrinkage*): mantener todas las variables pero **empujar los coeficientes hacia cero** de forma continua y controlada.

Formulación general:

$$\hat{\beta} = \arg \min_{\beta} \underbrace{\sum_{i=1}^n (y_i - x_i' \beta)^2}_{\text{ajuste}} + \lambda \cdot \underbrace{\mathcal{P}(\beta)}_{\text{penalización}}$$

- $\lambda \geq 0$ es la **perilla continua** de complejidad: $\lambda = 0$ es OLS; $\lambda \rightarrow \infty$ manda todos los coeficientes a cero. Se elige por **CV**: la curva de hace una hora, con λ en el eje.
- **Parámetros:** β . **Hiperparámetros:** λ (y α en Elastic Net). Hoy: tres penalizaciones $\mathcal{P}$, tres estimadores. Y la promesa de la sesión 1 en una frase: un poco de sesgo (encoger) compra mucha varianza cuando hay poca información por parámetro.

Ridge: encoger todo un poco

Problema (Hoerl & Kennard, 1970): $\mathcal{P}(\beta) = \sum_j \beta_j^2$.

$$\hat{\beta}^R = \arg \min_{\beta} \sum_{i=1}^n (y_i - x_i' \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad \Rightarrow \quad \hat{\beta}^R = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}'\mathbf{y}.$$

- La única diferencia con OLS es λ sumado a la diagonal (apéndice). Los autovalores de $\mathbf{X}'\mathbf{X} + \lambda \mathbf{I}$ son $d_j + \lambda > 0$: la inversa **siempre existe**, incluso con $p > n$, y las direcciones enfermas (d_j pequeño) son las que más se encogen.
- Con $\mathbf{X}$ ortonormal, $\hat{\beta}_j^R = \hat{\beta}_j^{OLS} / (1 + \lambda)$: encogimiento **proporcional**; ningún coeficiente llega exactamente a cero.
- **Sesgo-varianza** (ISL, Fig. 6.5): al subir λ , la varianza cae rápido y el sesgo sube despacio; siempre existe un $\lambda > 0$ mejor que OLS. Con la ficha y los 257 puestos: de 7,8 a 7,0 de CV.

Cuándo brilla

Muchos predictores con efectos pequeños y repartidos, o muy correlacionados entre sí: encoger todo un poco es mejor que apostar por unos cuantos.

Detalles que no son detalles: escala e intercepto

- **La penalización no es invariante a la escala:** $\sum_j \beta_j^2$ castiga igual al coeficiente de “personas por hogar” (de 2 a 5) que al de “estrato 1” (de 0 a 100). Si mido el estrato en proporción y no en porcentaje, su β se multiplica por cien y se penaliza de otra forma.
- **Solución: estandarizar** los predictores (media 0, desviación 1) antes de regularizar; así λ trata a todos por igual. Los coeficientes se reportan de vuelta en la escala original.
- **El intercepto no se penaliza:** β_0 mide el nivel promedio de y , no la complejidad del modelo. Encogerlo sesgaría todas las predicciones sin ganar nada.
- `glmnet` estandariza y excluye el intercepto **por defecto**; hay que saber que lo hace.

Regla de oro #3, otra vez

Las medias y desviaciones para estandarizar se calculan con los datos de entrenamiento **de cada pliegue**, no con toda la muestra. `cv.glmnet` lo hace solo: ajusta `glmnet`, y con él la estandarización, dentro de cada pliegue.

Lasso: un valor absoluto que cambia todo

Problema (Tibshirani, 1996): $\mathcal{P}(\beta) = \sum_j |\beta_j|$.

$$\hat{\beta}^L = \arg \min_{\beta} \sum_{i=1}^n (y_i - x_i' \beta)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- El único cambio frente a Ridge: $|\beta_j|$ en lugar de β_j^2 . La consecuencia es enorme: para λ suficientemente grande, algunos coeficientes son **exactamente cero**.
- El Lasso hace **dos trabajos a la vez**: encoge (control de varianza) y selecciona variables (modelos esparsos, más fáciles de leer). De los 37 predictores se quedó con **13**; con la regla 1-SE, con **6**.
- No tiene solución cerrada (el valor absoluto no es diferenciable en cero): `glmnet` lo resuelve coordenada por coordenada en milisegundos (algoritmo, condición de optimalidad y $\lambda_{\max}$ en el apéndice).

Lasso: un valor absoluto que cambia todo

Problema (Tibshirani, 1996): $\mathcal{P}(\beta) = \sum_j |\beta_j|$.

$$\hat{\beta}^L = \arg \min_{\beta} \sum_{i=1}^n (y_i - x_i' \beta)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

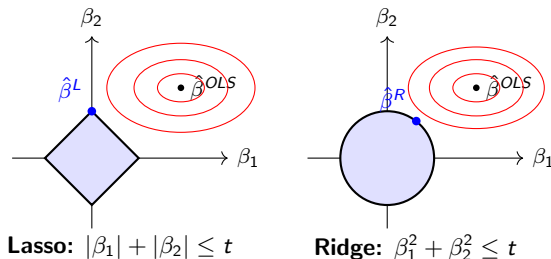
- El único cambio frente a Ridge: $|\beta_j|$ en lugar de β_j^2 . La consecuencia es enorme: para λ suficientemente grande, algunos coeficientes son **exactamente cero**.
- El Lasso hace **dos trabajos a la vez**: encoge (control de varianza) y selecciona variables (modelos esparsos, más fáciles de leer). De los 37 predictores se quedó con **13**; con la regla 1-SE, con **6**.
- No tiene solución cerrada (el valor absoluto no es diferenciable en cero): `glmnet` lo resuelve coordenada por coordenada en milisegundos (algoritmo, condición de optimalidad y $\lambda_{\max}$ en el apéndice).

¿Por qué el valor absoluto produce ceros exactos?

La respuesta está en la **geometría** de la restricción.

La geometría: por qué el Lasso anula y Ridge no

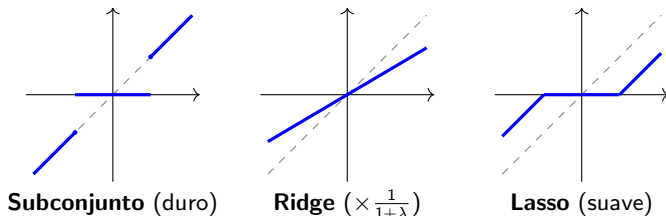
Forma equivalente (restringida): minimizar RSS sujeto a $\mathcal{P}(\beta) \leq t$.



- La solución está donde las curvas de nivel del RSS (elipses) **tocan** la región factible (azul).
- El diamante tiene **esquinas sobre los ejes** ($\beta_j = 0$): las elipses suelen tocar ahí primero $\Rightarrow$ ceros exactos. El círculo no tiene esquinas: el contacto casi nunca cae sobre un eje.

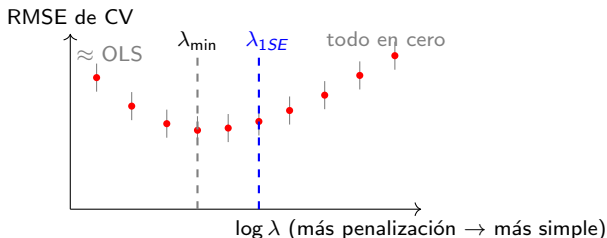
Los tres umbrales, cara a cara

Con $\mathbf{X}$ ortonormal, cada método transforma $\hat{\beta}_j^{OLS}$ así:



- **Subconjunto** = umbral **duro**: conserva intactos los grandes y elimina los chicos, con un salto: la inestabilidad de la selección.
- **Ridge** = encogimiento proporcional: nunca llega a cero.
- **Lasso** = umbral **suave**: $\hat{\beta}_j^L = \text{sign}(\hat{\beta}_j) (|\hat{\beta}_j| - \lambda)_+$; corre todo hacia cero y corta. Continuo y esparso: lo mejor de los dos mundos (la derivación, en el apéndice). El precio: los que sobreviven quedan **encogidos**, sesgados hacia cero a propósito.

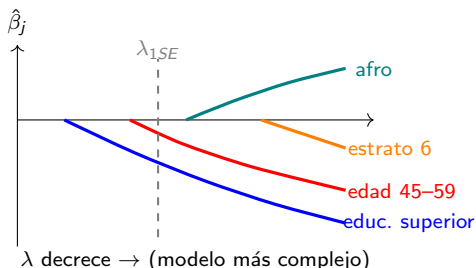
Elegir λ por CV: la curva de hace una hora, con λ en el eje



- `glmnet` ajusta el **camino completo** desde $\lambda_{\max}$ hacia abajo (unos 100 valores en escala logarítmica), usando cada solución como arranque de la siguiente. La curva entera cuesta poco más que un OLS.
- Con k pliegues, cada punto trae su barra. $\lambda_{\min}$ minimiza; λ_{1SE} es el modelo más simple indistinguible del mejor. Con la ficha y los 257 puestos: $\lambda_{\min} = 0,38$ deja 13 coeficientes y $\lambda_{1SE} = 1,83$ deja 6 (educación superior, tres edades, edad de votar y estrato 6): CV de 7,1 y 7,5.
- A la izquierda, con $\lambda \rightarrow 0$, la curva sube: es OLS sobreajustando (7,8). A la derecha, todo en cero: el modelo predice la media (12,2).

Leer un camino de regularización

La otra salida de `glmnet`: los coeficientes como función de λ .



- De izquierda (λ grande: todo en cero) a derecha ($\lambda \rightarrow 0$: OLS). Cada línea es un coeficiente; el punto donde se despega del cero es su **entrada**. (El esquema es ilustrativo; el camino real sale de `plot(glmnet)`.)
- **El orden de entrada informa:** lo que entra primero es lo que más poder predictivo aporta *dado lo que ya está*. Con la ficha entra primero la educación superior; después las edades (0-19, 45-59, 60-74), la edad de votar y el estrato 6; más tarde la población afro y la distancia. (El camino de Ridge es parecido, pero nadie toca el cero.)

¿Ridge o Lasso? Depende de cómo sea el mundo. Elastic Net: la negociación

- **Mundo esparso** (pocos coeficientes grandes, el resto ≈ 0): gana el **Lasso**, que recorta el ruido a cero.
- **Mundo denso** (muchos efectos pequeños repartidos): gana **Ridge**; anular variables tira señal a la basura. Con los 257 puestos, Ridge tiene el mejor CV (7,0).
- **Predictores muy correlacionados** (educación superior, estrato 6, internet): el Lasso elige **uno** del grupo, casi al azar $\Rightarrow$ selección **inestable**.

Elastic Net (Zou & Hastie, 2005), en la parametrización de `glmnet`:

$$\hat{\beta}^{EN} = \arg \min_{\beta} \frac{1}{2n} \sum_i (y_i - x_i' \beta)^2 + \lambda \left[\alpha \sum_j |\beta_j| + \frac{1-\alpha}{2} \sum_j \beta_j^2 \right], \quad \alpha \in [0, 1].$$

$\alpha = 1$ es Lasso; $\alpha = 0$, Ridge. Hereda del L1 la **selección** y del L2 la **estabilidad**: los predictores correlacionados entran o salen **juntos**. Dos hiperparámetros, (λ, α) : CV sobre ambos, o fijar $\alpha \in \{0, 0.5, 1\}$.

¿Ridge o Lasso? Depende de cómo sea el mundo. Elastic Net: la negociación

- **Mundo esparso** (pocos coeficientes grandes, el resto ≈ 0): gana el **Lasso**, que recorta el ruido a cero.
- **Mundo denso** (muchos efectos pequeños repartidos): gana **Ridge**; anular variables tira señal a la basura. Con los 257 puestos, Ridge tiene el mejor CV (7,0).
- **Predictores muy correlacionados** (educación superior, estrato 6, internet): el Lasso elige **uno** del grupo, casi al azar $\Rightarrow$ selección **inestable**.

Elastic Net (Zou & Hastie, 2005), en la parametrización de `glmnet`:

$$\hat{\beta}^{EN} = \arg \min_{\beta} \frac{1}{2n} \sum_i (y_i - x_i' \beta)^2 + \lambda \left[\alpha \sum_j |\beta_j| + \frac{1-\alpha}{2} \sum_j \beta_j^2 \right], \quad \alpha \in [0, 1].$$

$\alpha = 1$ es Lasso; $\alpha = 0$, Ridge. Hereda del L1 la **selección** y del L2 la **estabilidad**: los predictores correlacionados entran o salen **juntos**. Dos hiperparámetros, (λ, α) : CV sobre ambos, o fijar $\alpha \in \{0, 0.5, 1\}$.

En la práctica

Se prueban los tres sobre los **mismos pliegues** y decide el CV. Ridge = estabilidad; Lasso = esparsidad; Elastic Net = negociación. Y todo vale para la logística: `glmnet` con familia `binomial` es la logística penalizada de la sesión 4.

Predecir bien no es seleccionar bien

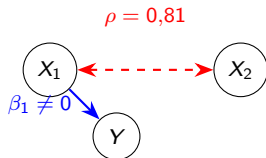
Supongamos que el mundo es $\mathbf{y} = \mathbf{X}\beta^0 + \varepsilon$ con β^0 **esparso**: sólo $s \ll n$ coeficientes distintos de cero (el supuesto que hace posible aprender con $p > n$). Dos preguntas que suenan igual y no lo son:

- ① **¿Predice bien?** ¿Es pequeño el error fuera de muestra de $\mathbf{X}\hat{\beta}$?
- ② **¿Selecciona bien?** ¿Recupera el soporte: $\{j : \hat{\beta}_j \neq 0\} = \{j : \beta_j^0 \neq 0\}$?
- Para (1) basta la esparcidad y un diseño no degenerado: con $\lambda \asymp \sigma \sqrt{\log(p)/n}$, el Lasso predice casi tan bien como OLS sobre las s variables correctas, pagando sólo un factor $\log p$ por no saber cuáles son (apéndice).
- Para (2) hacen falta, además, señales no minúsculas y la **condición de irrepresentabilidad**: que ningún predictor irrelevante sea demasiado bien explicado por los relevantes. Es fuerte, **no verificable**, y si falla el Lasso incluye proxies *sin importar cuántos datos haya*.
- La (2) es **mucho** más exigente que la (1): se puede predecir de maravilla con las variables “equivocadas”. En nuestra ficha, el estrato alto predice casi igual que la educación superior.

Y son relevantes para usos distintos

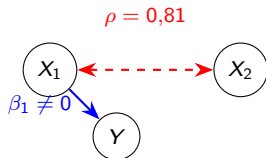
Para **predecir**, basta (1). Para **interpretar** qué variables importan, o para elegir **controles**, quisiéramos (2), y casi nunca la tenemos.

El modo de falla típico: el proxy correlacionado



- Supongan que X_1 (la educación superior) mueve el voto y X_2 (el estrato alto) es un proxy que no: en nuestros 257 puestos su correlación es 0,81. Para el Lasso ambos predicen casi igual: puede quedarse con X_2 , repartir, o alternar entre muestras.
- **En 20 remuestras de los 257 puestos**, el Lasso conservó la educación superior en el 100%, la edad de 45 a 59 en el 80%, la de 60 a 74 en el 70%, la población afro y el estrato 6 en el 60%, el estrato 4 y la distancia en el 50%. Media lista cambia con la muestra.
- Para **predecir**, da lo mismo. Para **interpretar**, es un desastre silencioso.

El modo de falla típico: el proxy correlacionado



- Supongan que X_1 (la educación superior) mueve el voto y X_2 (el estrato alto) es un proxy que no: en nuestros 257 puestos su correlación es 0,81. Para el Lasso ambos predicen casi igual: puede quedarse con X_2 , repartir, o alternar entre muestras.
- **En 20 remuestras de los 257 puestos**, el Lasso conservó la educación superior en el 100%, la edad de 45 a 59 en el 80%, la de 60 a 74 en el 70%, la población afro y el estrato 6 en el 60%, el estrato 4 y la distancia en el 50%. Media lista cambia con la muestra.
- Para **predecir**, da lo mismo. Para **interpretar**, es un desastre silencioso.

La frase para tatuarse

El Lasso selecciona **predictores**, no **causas**. El *stepwise* hereda el problema sin las garantías. Si lo que quieren es un β , seleccionen dos veces (apéndice A9).

- 1 Elegir sin gastar la prueba: la validación cruzada
- 2 El *pipeline* honesto: preprocesar dentro de cada pliegue
- 3 Cuando el k -fold aleatorio miente: grupos, espacio y tiempo
- 4 Muchos predictores: Ridge, Lasso y Elastic Net
- 5 Aplicación guiada en R
- 6 Cierre

Aplicación guiada (I): 37 predictores para 257 puestos

El área metropolitana de la semana 2, la misma partición, la ficha completa
(applications/week-03)

Modelo	Coef. $\neq 0$	RMSE dentro	RMSE de CV	RMSE de prueba
Recta (educación superior)	1	7,31	7,43	6,83
Las nueve variables	9	6,85	7,38	6,76
OLS con la ficha	37	5,94	7,76	6,70
Ridge, $\lambda_{\min}$	37	6,25	7,01	6,53
Elastic Net ($\alpha = 0,5$), $\lambda_{\min}$	18	6,47	7,08	6,63
Lasso, $\lambda_{\min}$	13	6,47	7,10	6,59
Lasso, λ_{1SE}	6	7,21	7,50	7,01

- **De conteos a predictores:** el script convierte el censo crudo en 37 proporciones, razones y logaritmos; ninguna aprende de los datos, así que se construyen antes de partir.
- **OLS con todo** tiene el mejor ajuste dentro ($R^2 = 0,76$; 8 de 37 coeficientes significativos) y el *peor* CV. Las penalizadas tienen el mejor CV sin que nadie les diga qué sobra.
- **El modelo parsimonioso:** con seis variables (Lasso, λ_{1SE}) el CV es 7,5, igual que la recta; con trece, 7,1. “¿Cuántas variables?” tiene respuesta, y la da la curva.
- **La prueba** (86 puestos) no distingue nada: el intervalo *bootstrap* de la diferencia entre la ficha y la recta va de $-1,1$ a $0,8$.

Aplicación guiada (II): el país y la ficha larga

9.603 puestos para aprender, 2.398 bajo llave: la partición fija del curso

Modelo	Coef.	Dentro	CV	Prueba
Media del entrenamiento	0	27,3		26,7
Recta (educación superior)	1	27,3		26,7
Las nueve variables	9	23,5		22,9
La ficha de 37	37	20,2		19,8
La ficha + departamentos	69	15,6	15,8	15,6
OLS con la ficha larga (ficha $\times$ departamento)	1.253	12,3		118,3
Ridge con la ficha larga, $\lambda_{\min}$	1.253		15,3	14,6
Lasso con la ficha larga, $\lambda_{\min}$	191		14,9	14,7

- **La recta de la semana 2 no sirve en el país** (26,7, igual que la media); la ficha baja a 19,8 y los departamentos a 15,6: la geografía pesa tanto como el censo.
- **La ficha larga** deja que cada predictor pese distinto en cada departamento: es lo que la recta de Cali necesitaba. OLS con 1.253 columnas tiene el mejor error dentro y **explota** en la prueba: en Vaupés hay 25 puestos para 38 coeficientes.
- **El Lasso** se queda con 191 de las 1.253 y llega a 14,7; la ventaja sobre la ficha con departamentos ($-0,9$; intervalo de $-1,1$ a $-0,7$) es real. Ridge empata en la prueba; el CV eligió al Lasso.

Aplicación guiada (III): el *pipeline* honesto, en tres tablas

La trampa del promedio municipal

	RMSE
CV, columna mal hecha	11,3
CV, hecha en el pliegue	13,3
Prueba	13,0

Vecinos que se copian

RMSE de CV	Al azar	Por mpio.
k -NN en el mapa	12,3	15,3
Ficha + dptos.	15,8	17,1

En el tiempo (RMSE en la prueba de 2022): modelo entrenado en 2022, 16,1; entrenado en 2018, 19,9; el de 2018 corrigiendo el nivel, 17,8.

- **Los vecinos del mapa** son hoy el mejor predictor del país: 12,1 en la prueba, sin usar una sola variable del censo. Pero responden otra pregunta: completar el mapa donde ya hay puestos vecinos. En municipios nuevos valen 15,3, y no dicen nada sobre *por qué* un territorio vota como vota.
- **La estructura viaja en el tiempo a medias:** el modelo de 2018 conserva parte de su poder en 2022, y lo que más falla es el nivel.

- 1 Elegir sin gastar la prueba: la validación cruzada
- 2 El *pipeline* honesto: preprocesar dentro de cada pliegue
- 3 Cuando el k -fold aleatorio miente: grupos, espacio y tiempo
- 4 Muchos predictores: Ridge, Lasso y Elastic Net
- 5 Aplicación guiada en R
- 6 Cierre

Qué dice el modelo y qué no dice

- **Un número de CV estima el error de un procedimiento**, no de un modelo concreto. Por eso es algo pesimista, por eso se reajusta con todo al final, y por eso el CV con el que se eligió es optimista para el elegido: el número que se reporta sale de la prueba.
- **Responde la pregunta de su esquema**: un k -NN con RMSE de 12,3 en CV al azar y de 15,3 por municipio no es una contradicción: son dos usos distintos del modelo.
- **Los coeficientes de Ridge y Lasso están encogidos**: un $\hat{\beta}_j^L$ no es un efecto marginal; está sesgado hacia cero a propósito. Si se quiere la magnitud, se reajusta OLS sobre las variables que sobrevivieron (*post-Lasso*), cuyo error estándar **finge** que el modelo estaba decidido de antemano.
- **Sobrevivir no es importar**: lo que sobrevive es lo que predice *dado lo demás*; con proxies correlacionados el sorteo cambia entre remuestras, y la tabla de estabilidad es el mínimo honesto.

Y , no β , versión sesión 3

En Inferencia Causal la especificación se defiende con teoría e identificación; aquí, con el *pipeline*: cómo se partió, cómo se afinó, dónde se midió. Que la educación superior entre primera al camino no dice que educar a un barrio cambie su voto.

Cuándo usar qué, y qué leer

	Usar cuando	Cuidado con
<i>k</i> -fold aleatorio	datos tabulares sin estructura; $k = 5$ o 10, estratificado	repetirlo si n es pequeño
Por grupos o bloques	unidades agrupadas, datos espaciales	quitar las variables que identifican el grupo
Hacia adelante	series de tiempo, otra elección	nunca barajar
Regla 1-SE	elegir hiperparámetros; reportar un modelo esparso	el mejor CV entre cien es optimista
OLS	$p \ll n$ y especificación defendible	con p cerca de n , la varianza
Ridge	mundo denso; predictores correlacionados	no selecciona; estandarizar
Lasso	mundo esparso; se quiere un modelo legible	selección inestable; coeficientes encogidos
Elastic Net	grupos de predictores correlacionados	dos hiperparámetros por CV
<i>Stepwise</i>	casi nunca	<i>greedy</i> , inestable, sin garantías

Qué leer: ISL, cap. 5 (secs. 5.1–5.2) y cap. 6 (secs. 6.2 y 6.4, con las figuras de clase); Roberts et al. (2017) para el CV por bloques; Friedman, Hastie & Tibshirani (2010) si quieren saber qué hace `glmnet` por dentro.

Recap

- 1 **La prueba responde una pregunta y una sola vez.** Elegir modelos e hiperparámetros se hace **dentro del entrenamiento**: k -fold, la curva de CV, el mínimo o la regla 1-SE. Con los 257 puestos el CV eligió $k = 3$ (o 5) para los vecinos y descartó a OLS con la ficha.
- 2 **Regla de oro #3:** el CV valida *pipelines*. Todo lo que aprende de los datos (imputar, escalar, agregar, seleccionar, remuestrear) ocurre dentro de cada pliegue. El promedio municipal: un RMSE de 11,3 que en la prueba era 13,0.
- 3 **El k -fold aleatorio miente** con grupos, espacio y tiempo: validar por municipio o hacia adelante; cada esquema responde una pregunta distinta y hay que decir cuál. El *bootstrap* pone intervalos a cualquier métrica.
- 4 **Muchos predictores** = poca información por parámetro. **Ridge** encoge proporcionalmente y nunca anula; **Lasso** produce **ceros exactos** (el diamante, el umbral suave); **Elastic Net** negocia. λ se elige con la misma curva; los predictores se estandarizan dentro del pliegue.
- 5 **Predecir bien es fácil; seleccionar bien es casi imposible:** el Lasso selecciona *predictores*, no *causas*. Con 1.253 columnas evitó el desastre de OLS (118) y dejó el error del país en 14,7.

La tabla del líder

Desde hoy es nacional: 2.398 puestos de prueba, los mismos para todos

Sem.	Modelo	Ubicación	RMSE prueba
2	Media del entrenamiento	no	26,7
2	Recta (educación superior)	no	26,7
2	Las nueve variables	no	22,9
3	La ficha de 37	no	19,8
3	La ficha + departamentos	departamento	15,6
3	Ridge con la ficha larga	departamento	14,6
3	Lasso con la ficha larga (elegido por CV)	departamento	14,7
3	k -NN en el mapa ($k = 10$)	sólo la ubicación	12,1

- Dos ligas: la de la **estructura** (el censo y el departamento), que lidera el Lasso con 14,7, y la de la **ubicación**, donde los vecinos marcan 12,1. La campaña quiere la segunda; el analista, la primera.
- **La limitación que nos lleva a la sesión 5:** para que un modelo lineal entendiera que la educación pesa distinto en Cali y en el Pacífico le escribimos 1.253 columnas a mano. ¿No hay un método que encuentre las interacciones solo? Antes, la otra lectura del caso: la sesión 4.

Preparación para la próxima clase...

Para aprovechar al máximo la próxima sesión

Lecturas de esta semana:

- ISL, secciones 4.1–4.3 (clasificación y regresión logística) y 4.7.6 (logística penalizada, en el laboratorio).

Guía: la sección 4.4.2 tiene la matriz de confusión y la curva ROC con el ejemplo *Default*; las secciones 4.4–4.5 (LDA, QDA, Naive Bayes) van al apéndice de la sesión.

Próxima sesión — clasificación: de la logística a las decisiones. La misma elección, otra pregunta: a la campaña no le importa si Petro saca 48 o 52 en un puesto; le importa **si lo gana**. y pasa a ser una categoría; la logística como referencia y, con las herramientas de hoy, la logística penalizada y k -NN; y la mitad que decide: por qué la exactitud engaña, la matriz de confusión, el umbral como decisión con costos, ROC y AUC, calibración.

Aplicación en R: los 12.001 puestos, la ficha de hoy y una y nueva, `gana_petro`. **Se publica el Problem Set 1** (vivienda en Cali; entrega: martes 20 de octubre, 8:00).

Referencias esenciales

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. ed.). Springer. Caps. 5–6. [ISL]
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2. ed.). Springer. Sec. 7.10. [ESL]
- Roberts, D. R., et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8), 913–929.
- Neunhoeffler, M., & Sternberg, S. (2019). How Cross-Validation Can Go Wrong and What to Do About It. *Political Analysis*, 27(1), 101–106.
- Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *JMLR*, 13, 281–305.
- Hoerl, A., & Kennard, R. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
- Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *JRSS-B*, 58(1), 267–288.
- Zou, H., & Hastie, T. (2005). Regularization and Variable Selection via the Elastic Net. *JRSS-B*, 67(2), 301–320.
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization Paths for Generalized Linear Models via Coordinate Descent. *Journal of Statistical Software*, 33(1). [glmnet]
- Belloni, A., Chernozhukov, V., & Hansen, C. (2014). High-Dimensional Methods and Inference on Structural and Treatment Effects. *JEP*, 28(2), 29–50.
- Gelvez, J. D., Cardiles, A., Martínez-González, E. F., & Muñoz, M. (2026). How Predictable Is an Election? Working paper. [De donde salen los datos del caso.]

7 Apéndice: para profundizar

A1. El atajo de LOOCV en regresión lineal

Para mínimos cuadrados (y cualquier suavizador lineal) existe una identidad: con **un solo ajuste** sobre todos los datos,

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2,$$

donde h_i es el *leverage* de la observación i : el elemento i de la diagonal de $\mathbf{H} = \mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'$.

- **Intuición:** una observación con *leverage* alto “atrae” la recta hacia sí misma y su residuo dentro de muestra es artificialmente pequeño. Dividir por $(1 - h_i)$ deshace ese favor.
- El costo pasa de n ajustes a **uno**. Con $\sum_i h_i = p + 1$, el *leverage* promedio es $(p + 1)/n$: la corrección crece con la complejidad del modelo, como los criterios de información.

Alcance

Vale para OLS, *splines* y otros suavizadores lineales. Para árboles, *boosting* o redes no hay atajo: por eso en la práctica domina el k -fold.

A2. La cuenta del 0,632 y el estimador *out-of-bag*

1. Una observación fija i . En cada extracción con reemplazo, $P(\text{no sale } i) = 1 - 1/n$. Las n extracciones son independientes:

$$P(i \notin \text{remuestra}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0,368,$$

así que $P(i \in \text{remuestra}) \rightarrow 0,632$. Con $n = 100$ ya da 0,634: el límite es una excelente aproximación. **2. El estimador OOB.** Con B remuestras y un modelo $\hat{f}^{*b}$ por remuestra, para cada i se promedian sólo los modelos que no la vieron:

$$\widehat{\text{Err}}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n \frac{1}{|B_{-i}|} \sum_{b \in B_{-i}} L(y_i, \hat{f}^{*b}(x_i)),$$

con B_{-i} el conjunto de remuestras que no contienen a i .

- Cada $\hat{f}^{*b}$ se entrenó con cerca del 63% de las observaciones distintas: el OOB es algo pesimista, como un CV con $k \approx 3$. Para un bosque grande la diferencia con el CV de 10 pliegues es pequeña y el costo, nulo.
- ESL (sec. 7.11) propone el estimador “0,632”, que mezcla el error OOB con el de entrenamiento para corregir ese pesimismo.

A3. CV anidado y búsqueda bayesiana de hiperparámetros

CV anidado. Si probamos 100 configuraciones y reportamos el mejor CV, ese número es optimista: es el máximo de 100 variables ruidosas. Un lazo **externo** evalúa; dentro de cada pliegue de entrenamiento, un lazo **interno** afina. Lo que se reporta nunca se usó para elegir. En este curso usamos la versión sencilla: afinar con CV dentro del entrenamiento, reportar en la prueba. El anidado hace falta cuando **no hay** prueba aparte o cuando se quiere el error del *procedimiento* completo, con su afinación incluida.

Búsqueda bayesiana (Snoek, Larochelle & Adams, 2012). Cada evaluación de $CV(\theta)$ es cara (cinco ajustes de un bosque); queremos el mínimo con pocas evaluaciones:

- 1 Evaluar unas pocas configuraciones iniciales (al azar).
- 2 Ajustar un **modelo sustituto** de $CV(\theta)$ sobre lo evaluado (un proceso gaussiano), que da una media y una incertidumbre para cada θ no probado.
- 3 Elegir la siguiente θ maximizando una **función de adquisición** (la mejora esperada): alta donde la media predicha es baja o donde la incertidumbre es grande.
- 4 Evaluar, actualizar el sustituto, repetir hasta agotar el presupuesto.

En R: `tune::tune_bayes()`; con un bosque y cinco hiperparámetros, cien iteraciones suelen bastar.

A4. Ridge: la solución cerrada y la magia de $\lambda \mathbf{I}$

1. La función objetivo:

$$S(\beta) = (\mathbf{y} - \mathbf{X}\beta)'(\mathbf{y} - \mathbf{X}\beta) + \lambda \beta' \beta = \mathbf{y}'\mathbf{y} - 2\beta' \mathbf{X}'\mathbf{y} + \beta'(\mathbf{X}'\mathbf{X})\beta + \lambda \beta' \beta.$$

2. Derivamos e igualamos a cero:

$$-2\mathbf{X}'\mathbf{y} + 2\mathbf{X}'\mathbf{X}\beta + 2\lambda\beta = 0 \Rightarrow (\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})\beta = \mathbf{X}'\mathbf{y}.$$

3. Solución:

$$\hat{\beta}^R = (\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1} \mathbf{X}'\mathbf{y}.$$

4. Por qué la inversa siempre existe: si $d_1 \geq \dots \geq d_p \geq 0$ son los autovalores de $\mathbf{X}'\mathbf{X}$, los de $\mathbf{X}'\mathbf{X} + \lambda\mathbf{I}$ son $d_j + \lambda > 0$ para todo $\lambda > 0$, incluso con $p > n$. En la base de autovectores, Ridge multiplica la coordenada j de OLS por $d_j/(d_j + \lambda)$: casi 1 donde hay mucha varianza, casi 0 en las direcciones enfermas.

5. Caso ortonormal ($\mathbf{X}'\mathbf{X} = \mathbf{I}$): $\hat{\beta}_j^R = \hat{\beta}_j^{OLS}/(1 + \lambda)$, encogimiento proporcional.

Sesgo y varianza

$\mathbb{E}[\hat{\beta}^R] = (\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1} \mathbf{X}'\mathbf{X} \beta \neq \beta$: sesgado hacia cero.

$\text{Var}(\hat{\beta}^R) = \sigma^2 (\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1} \mathbf{X}'\mathbf{X} (\mathbf{X}'\mathbf{X} + \lambda\mathbf{I})^{-1}$, menor que la de OLS en cada dirección. Existe $\lambda > 0$ que reduce el MSE (Hoerl & Kennard, 1970).

A5. Las condiciones de optimalidad del Lasso

1. El obstáculo: $|\beta_j|$ no es diferenciable en cero, que es justo donde ocurre lo interesante. Se generaliza la derivada: g es un **subgradiente** de una función convexa f en x_0 si $f(x) \geq f(x_0) + g(x - x_0)$ para todo x . Para el valor absoluto,

$$\partial|\beta_j| = \begin{cases} \{\text{sign}(\beta_j)\} & \beta_j \neq 0 \\ [-1, 1] & \beta_j = 0 \end{cases}, \quad \text{y } \hat{\beta} \text{ minimiza } f \text{ convexa} \iff 0 \in \partial f(\hat{\beta}).$$

2. Aplicado a $\frac{1}{2}\|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda\|\beta\|_1$: el primer término tiene gradiente $-\mathbf{X}'(\mathbf{y} - \mathbf{X}\beta)$; el segundo aporta el subgradiente. Componente a componente:

$$\hat{\beta}_j \neq 0 : \mathbf{x}'_j(\mathbf{y} - \mathbf{X}\hat{\beta}) = \lambda \text{sign}(\hat{\beta}_j); \quad \hat{\beta}_j = 0 : |\mathbf{x}'_j(\mathbf{y} - \mathbf{X}\hat{\beta})| \leq \lambda.$$

- $\mathbf{x}'_j(\mathbf{y} - \mathbf{X}\hat{\beta})$ es la covarianza del predictor j con el **residuo**. Los activos la tienen exactamente en $\pm\lambda$; los inactivos, por debajo. λ es un **umbral de mérito**: entras sólo si tu correlación con lo aún no explicado alcanza λ .
- Si $\lambda \geq \max_j |\mathbf{x}'_j \mathbf{y}|$, $\hat{\beta} = 0$ cumple las condiciones: nadie entra. De ahí sale $\lambda_{\max}$.

A6. El umbral suave y el descenso por coordenadas

Caso univariado (una coordenada, $\mathbf{X}$ estandarizada): sea z el estimador sin penalizar; resolvemos $\min_{\beta} \frac{1}{2}(z - \beta)^2 + \lambda|\beta|$.

- 1 Si $\hat{\beta} > 0$: $\beta - z + \lambda = 0 \Rightarrow \hat{\beta} = z - \lambda$, coherente sólo si $z > \lambda$.
- 2 Si $\hat{\beta} < 0$: $\beta - z - \lambda = 0 \Rightarrow \hat{\beta} = z + \lambda$, coherente sólo si $z < -\lambda$.
- 3 Si $\hat{\beta} = 0$: se necesita $0 \in \{-z + \lambda s : s \in [-1, 1]\}$, es decir, $z \in [-\lambda, \lambda]$.

$$\hat{\beta} = S_{\lambda}(z) = \text{sign}(z) (|z| - \lambda)_+$$

Descenso por coordenadas (Friedman, Hastie & Tibshirani, 2010), el algoritmo de `glmnet`: con el objetivo $\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|_1$ y predictores estandarizados, inicializar $\hat{\beta} = 0$ y repetir, para $j = 1, \dots, p$, hasta converger:

$$r^{(j)} = \mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \hat{\beta}_k, \quad \hat{\beta}_j \leftarrow S_{\lambda}\left(\frac{1}{n} \mathbf{x}_j' r^{(j)}\right).$$

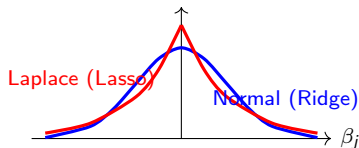
- Converge al óptimo global porque el objetivo es convexo y la parte no diferenciable es separable por coordenadas. Los trucos: *warm starts* a lo largo del camino de λ y *active sets* (iterar sólo sobre los activos). Por eso el camino completo cuesta poco más que un OLS.

A7. La lectura bayesiana: penalizar es tener un *prior*

Ridge y Lasso son **modas posteriores** bajo distintos *priors* independientes sobre los β_j :

$$\text{Ridge: } \beta_j \sim N(0, \tau^2)$$

$$\text{Lasso: } \beta_j \sim \text{Laplace}(0, b)$$



- Con $\varepsilon \sim N(0, \sigma^2)$, maximizar la posterior es minimizar $\text{RSS} + \lambda \mathcal{P}(\beta)$ con $\lambda = \sigma^2 / \tau^2$ (Ridge) o $\lambda = 2\sigma^2 / b$ (Lasso).
- El *prior* de Laplace es **picudo en cero**: apuesta a que muchos coeficientes son exactamente nulos, y la moda posterior produce ceros. λ traduce la fuerza de la creencia.
- Mensaje: la penalización no es un truco numérico; es **información previa** sobre el mundo (“los efectos son pequeños” o “son pocos”). Por eso el CV, que la elige con datos, es lo que la hace honesta.

A8. Lo que la teoría del Lasso garantiza (y lo que exige)

Sea $\mathbf{y} = \mathbf{X}\beta^0 + \varepsilon$ con β^0 esparso de soporte S , $|S| = s$.

Resultado 1 (predicción). Con $\lambda \asymp \sigma \sqrt{\log(p)/n}$ y un diseño no degenerado en las direcciones esparsas (condición de *autovalor restringido*),

$$\frac{1}{n} \|\mathbf{X}(\hat{\beta}^L - \beta^0)\|_2^2 \lesssim \sigma^2 \frac{s \log p}{n}.$$

Si conociéramos las s variables correctas, OLS sobre ellas daría $\sigma^2 s/n$: el costo de la búsqueda es sólo $\log p$ (con $p = 10^6$, $\log p \approx 14$). La condición es mucho más débil que pedir $\mathbf{X}'\mathbf{X}$ invertible; por eso el Lasso vive donde OLS muere.

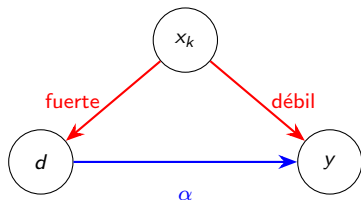
Resultado 2 (selección). Para recuperar el soporte con alta probabilidad hacen falta, además, (i) $\min_{j \in S} |\beta_j^0| \gtrsim \lambda$ (señales no minúsculas) y (ii) la **condición de irrepresentabilidad** (Zhao & Yu, 2006):

$$\|\mathbf{X}'_{S^c} \mathbf{X}_S (\mathbf{X}'_S \mathbf{X}_S)^{-1} \text{sign}(\beta_S^0)\|_\infty < 1,$$

es decir, ningún predictor irrelevante puede ser demasiado bien explicado por los relevantes. Es fuerte, no verificable, y si falla el Lasso incluye proxies sin importar n .

Referencias: Hastie, Tibshirani & Wainwright (2015), *Statistical Learning with Sparsity*, caps. 2 y 11; Wainwright (2019), *High-Dimensional Statistics*, caps. 7–8.

A9. Si lo que quieren es un β : seleccionen dos veces



Setup: el efecto α de un tratamiento d sobre y , controlando flexiblemente por cientos de x . **La tentación:** Lasso de y sobre (d, x) para elegir controles, y OLS con los elegidos.

- **El fallo:** el Lasso elige por capacidad de predecir y . Un confusor con efecto *débil* sobre y pero *fuerte* sobre d es descartado, y su omisión sesga $\hat{\alpha}$: el sesgo de variable omitida, resucitado por un algoritmo.

Post-double-selection (Belloni, Chernozhukov & Hansen, 2014)

Lasso de y sobre x y Lasso de d sobre x ; OLS de y sobre d y la **unión** de los controles seleccionados. Para sesgar $\hat{\alpha}$ una variable omitida debe correlacionar con y y con d ; la doble selección sólo omite las débiles en ambas, y el sesgo residual es de segundo orden. Los intervalos usuales vuelven a ser válidos. Es el puente con Inferencia Causal que retomamos al cierre del curso.